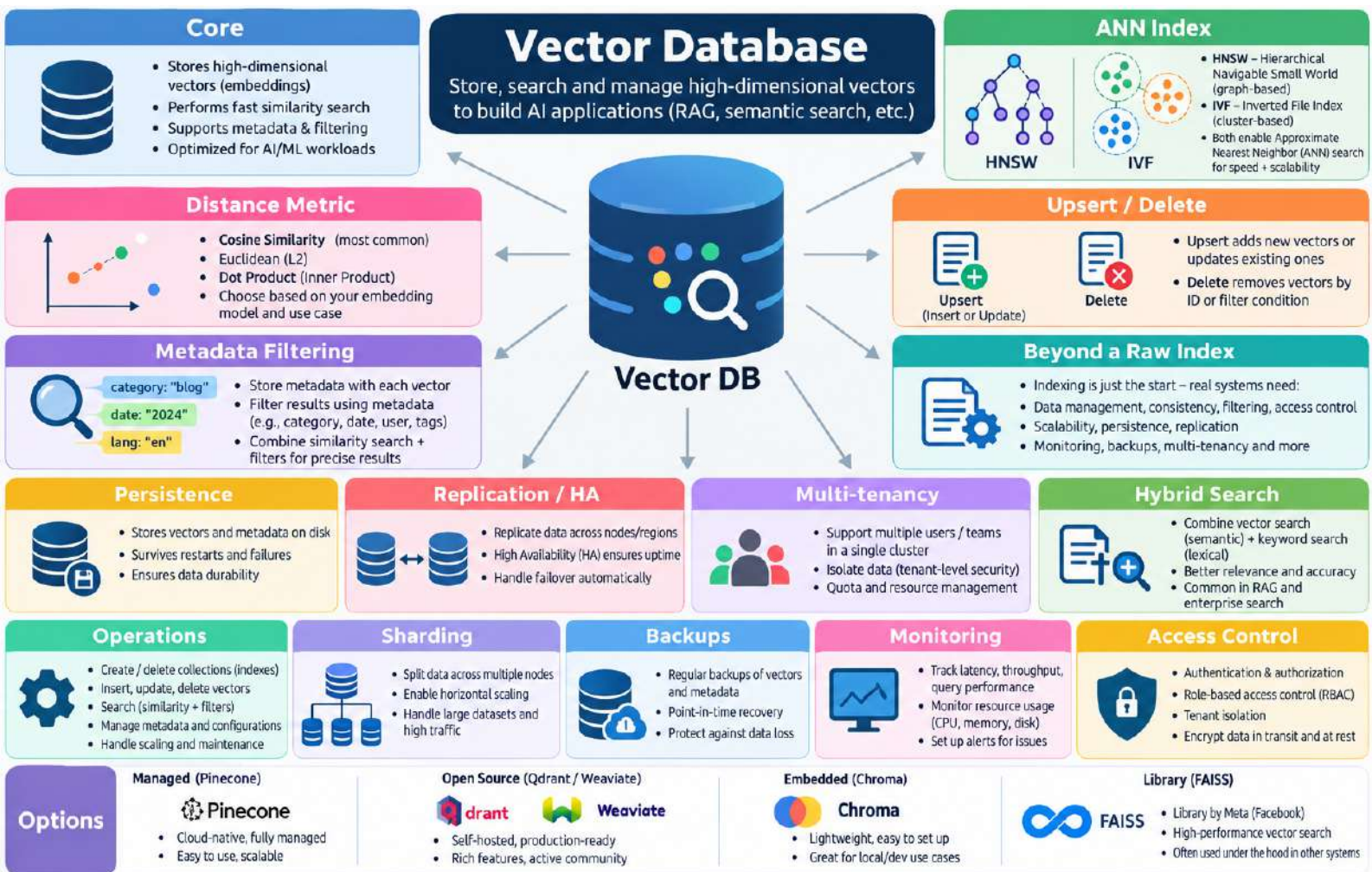




From Vectors to Value — Powering the Next Generation of AI Applications!

Same Vector Search Idea. Very Different Capabilities.

An ANN library is great for experiments. A Vector Database is built for real-world applications.



Raw ANN library (in-memory)

e.g., FAISS (as a library), hnswlib, Annoy



A library you use in your application to build an in-memory index for approximate nearest neighbor (ANN) search.



No persistence

Index lives only in memory (lost on restart)



No metadata filter

Usually only vector search (no structured filters like user, category, date, etc.)



No scaling / HA

Single machine, no built-in sharding, replication or high availability



No auth / multi-tenant

You need to build everything yourself

Great for research, testing and small prototypes, but not production-ready.

VS



Vector Database

e.g., Pinecone, Qdrant, Weaviate, Chroma (persistent), etc.



A purpose-built system to store, manage and search vector embeddings at scale, with production features beyond a raw ANN index.



Durable storage

Persists vectors and metadata on disk (survives restarts and failures)



Filter + search together

Search vectors with metadata filters (e.g., user, category, date, tags)



Shard / replicate

Built-in horizontal scaling, replication and high availability (HA)



APIs, auth, backups

Production-ready APIs, access control, multi-tenancy, backups and monitoring

Built for real-world, scalable, multi-tenant applications.



Use an ANN library to learn. Use a Vector Database to ship.



From Prototype to Production



Why a Dedicated Vector Database?

You could try to DIY... but here's what happens vs. using a real Vector DB.

1 Store vectors in a plain SQL/array



documents

id	text	embedding (array)
1	Cat	[0.12, 0.83, ...]
2	Dog	[0.04, 0.56, ...]
3	Car	[0.33, 0.21, ...]
...	...	...



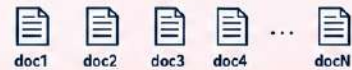
Works for small data, but doesn't scale.

Storing high-dimensional vectors in a regular SQL table is inefficient.

2 Scan ALL rows per query



For each row:
compute_distance(query, row.embedding)
sort and return top-k



O(N) — seconds at millions

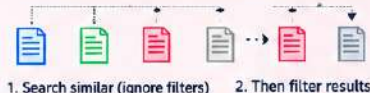
You have to compute distance with every vector. This becomes very slow as data grows.

3 No metadata filter + ANN combo

You want:



Find similar documents
where category = 'blog'



Filter after search → wrong top-k

If you search first and filter later, you might miss the real top-k results because filtered items were not considered during search.



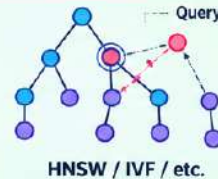
Use a Vector Database



- ✓ Designed to store and index high-dimensional vectors
- ✓ Optimized storage (compression, quantization, etc.)
- ✓ Fast and scalable for millions/billions of vectors
- ✓ Built-in APIs and tooling



Use a Vector Database with ANN Index



HNSW / IVF / etc.

- ✓ Uses Approximate Nearest Neighbor (ANN) indexes (e.g., HNSW, IVF)
- ✓ Searches only a small part of the index
- ✓ Returns top-k results in **milliseconds**
- ✓ Scales to millions/billions of vectors



Use a Vector Database with Metadata Filtering



Find similar documents
where category = 'blog'
AND date >= '2024-01-01'



Filter + ANN together
→ correct top-k

- ✓ Filter and search in a **single query**
- ✓ Uses metadata filters (e.g., category, date, user, tags)
- ✓ Returns accurate top-k results
- ✓ Built-in support for hybrid search (vector + keyword + filters)



Bottom Line:

You can DIY for learning or tiny datasets, **but for real-world applications, use a Vector Database.**



Which Vector Database / Tool Should You Use?

Different needs. Different tools. Find the best fit for your use case.

What's your situation?



Need fully managed cloud, no ops team



Fully managed vector database in the cloud.

Pay for zero-ops + scale

Want open-source + great performance / filtering



Open-source vector database with high performance and powerful filtering.

Self-host or Qdrant Cloud

Want OSS + hybrid search + built-in modules



Open-source vector database with hybrid search (keyword + vector) and rich built-in modules (e.g., generative, rerankers).

Keyword + vector in one engine

Local prototype / small app, fast start



Lightweight, easy-to-use vector database for prototyping and small applications.

pip install, embed, done

Library-level control, in-process, custom infra



A high-performance similarity search library (by Meta) for in-memory vector search.

Wrap it yourself for prod

	Pinecone	Qdrant	Weaviate	Chroma	FAISS
Type	Cloud Vector Database	Vector Database	Vector Database	Embedded Vector DB	ANN Library
Managed Service	✓ Yes (fully managed)	✓ (Qdrant Cloud)	— (Cloud option available)	✗ No	✗ No
Open Source	✗ No (proprietary)	✓ Yes	✓ Yes	✓ Yes	✓ Yes
Metadata Filtering	✓ Yes	✓ Yes (powerful)	✓ Yes	✓ Yes	✗ No (basic only)
Hybrid Search (keyword + vector)	✗ Not built-in	— Possible (with integrations)	✓ Yes (built-in)	— Possible (with integrations)	✗ No
Best For	Production, large scale, no ops	High performance, self-hosted or cloud	Hybrid search, RAG apps, rich modules	Prototyping, small apps, quick start	Custom systems, full control

Bottom Line: There's **no single best tool** — it depends on your **use case, team, and constraints**.

→ Pick the tool that fits your journey!

Common Triggers (You probably need a Vector Database)



Store millions of embeddings



Fast semantic search



Filter by tenant/date and similarity



RAG at scale



Persistent + highly available



If they say "small dataset" or "just a prototype", propose **Chroma** / **FAISS** / **pgvector** instead of over-engineering.

Which Vector Solution Should You Use?

Answer a few questions to find the best fit.

What's the size of your data?

Small dataset
(≤ 100K embeddings)
or just a prototype

Large dataset
(millions of embeddings
or more)

Do you need a fully managed cloud solution (no ops team)?

Yes

No

Do you need hybrid search (keyword + vector) or rich modules (e.g., generators, rerankers)?

Yes

No

Do you want open-source with great performance and filtering?

Yes

No

Need library-level control, in-process, or custom infra?

Keep it simple
Use **Chroma** / **FAISS** / **pgvector**

Chroma



Chroma

- Local prototype
- Small apps / datasets
- Easy to use
- pip install, embed, done

FAISS



FAISS

- Library level (by Meta)
- High performance
- In-memory, in-process
- Wrap it yourself for prod

Pinecone



Pinecone

- Fully managed (SaaS)
- Zero ops
- Scales to billions
- Pay for convenience + scale

Weaviate



Weaviate

- Open source
- Hybrid search (keyword + vector)
- Rich built-in modules (generative, rerankers, etc.)
- Self-host or managed

Qdrant



Qdrant

- Open source
- High performance
- Excellent filtering (metadata)
- Self-host or Qdrant Cloud
- Great for real-world use cases



Recommended Option



Key Takeaway: Use a dedicated Vector Database when you need scale, speed, filtering, and reliability for real-world applications (like RAG). For small datasets or quick prototypes, Chroma / FAISS / pgvector are great starting points.



"Start simple.
Scale when needed."



Vector Database – Core Mental Model

From raw data to meaningful results — how it all fits together

Same concept
Powering Search,
RAG and Recommendations!

1 Documents
Your raw data
PDFs, web pages, text files, databases, images, etc.



- Examples**
- PDF manuals
 - Support tickets
 - Web articles
 - Product catalog
 - Images (with captions)

2 Chunk
Split into smaller pieces
Break documents into meaningful chunks (e.g., 200–500 tokens).



- Why?**
- Smaller chunks = better context
 - Improves retrieval accuracy
 - Handles large documents

3 Embed
Convert text to vectors
Use an embedding model (e.g., OpenAI, BGE, E5, etc.)

"This is a document chunk...." → [0.12, -0.34, 0.56, ...]

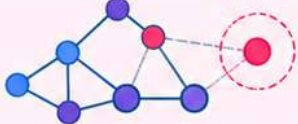
- Result**
- High-dimensional vector
 - Captures semantic meaning
 - Same model for queries

4 Vector + Metadata
Store in Vector DB
Save both the vector and useful metadata (e.g., source, date, tenant).

Vector:	[0.12, -0.34, 0.56, ...]
Metadata:	{ "source": "blog.pdf", "date": "2024-01-15", "tenant": "acme", "category": "tech" }

- Why metadata?**
- Enables filtering (tenant, date, tag)
 - Improves relevance
 - Needed for real-world use cases

5 ANN / HNSW
Fast similarity search
Use an approximate nearest neighbor index (e.g., HNSW, IVF) for sub-second search at scale.



Find nearest vectors (approximate)

- Why ANN?**
- Exact search is too slow ($O(N)$)
 - HNSW/IVF gives very fast results
 - Scales to millions/billions of vectors

6 Filtered Top-K
Apply metadata filters
Combine similarity search with filters (e.g., tenant = X, date >= 2024-01-01).



Top-K relevant chunks (after filtering)

1. ...
2. ...
3. ...

- Example filters**
- tenant = "acme"
 - date >= "2024-01-01"
 - category = "product"
 - user_id = "123"

7 Reranker
Improve relevance
Use a cross-encoder or reranking model to refine the top-K results.



1. (best) ... 
2. ...
3. ...

- Why rerank?**
- ANN is fast but approximate
 - Reranker improves accuracy
 - Gives you the most relevant results

8 RAG / Search / Reco
Use the results
Feed the top results to your application (e.g., LLM for RAG, search UI, or recommender).



RAG (LLM + your data) **Search** (semantic search) **Recommendations** (similar items)

- Real-world applications**
- Chat with your documents (RAG)
 - Enterprise search
 - Product/content recommendations
 - Knowledge management



Key Takeaway: A Vector DB doesn't just store vectors — it enables semantic search at scale with metadata, filtering, and production-ready features.



From data
to real value!

Pre-Filter vs Post-Filter in Vector Search

Same goal: Top-10 similar docs WHERE tenant = A. But the approach makes a big difference!

Filter early.
Get better results!

Suppose:



Tenant A
100 vectors



Tenant B
900 vectors

Query:



Top-10 similar docs WHERE tenant = A

❌ Post-filter (Filter after search)

❌ Can miss good results

Search globally first, then filter the results.

1

ANN search on
ALL vectors
(Tenant A + Tenant B)

Q Query



Top 10 (global)

- B - doc1
- B - doc2
- B - doc3
- A - doc4
- B - doc5
- ...

2

Remove Tenant B
from results

After filtering (only Tenant A)

- A - doc4
- ...

3

Maybe only 1 valid
result (less than 10)

You wanted 10 results, but got only 1
because most of the top results were from Tenant B.



Problem: You can miss relevant results
because the initial top-k didn't consider the filter.

✅ Pre-Filter / Integrated Filtering

✅ Correct & consistent results

Apply the filter first (or during search), then run ANN on allowed candidates.

1

Apply filter
(tenant = A)

→ Only search in
allowed candidates

Query
+ tenant = A



Top 10 (Tenant A)

- A - doc1
- A - doc2
- A - doc3
- A - doc4
- ...
- A - doc10

2

ANN search within
filtered set

Search only in Tenant A's
vectors using ANN (e.g., HNSW, IVF)
with metadata filter.

3

Get Top-10
Tenant-A results



You get 10 relevant results,
all from Tenant A.



Advantage: Faster, more accurate, and always gives
you the right top-k for the filtered set.



Key Takeaway:

Use **pre-filtered / integrated filtering** in a Vector Database,
especially when you need to combine metadata filters (e.g., tenant, date, tags) with similarity search.



Right data.
Right results.



Scaling a Vector Database

From thousands to billions of vectors — what matters and how to scale

Same concepts.
Bigger possibilities.

1 Key Dimensions (Capacity)

These factors determine how much data you can store and how much compute/memory you need.



$$\text{Total Storage/Memory} \approx N \times d \times \text{index_overhead} \times R$$

2 Other Important Factors

These affect performance and real-world scalability.

- QPS (Queries per second)**
How many searches per second?
- Write throughput**
How many vectors inserted/updated per second?
- Filter selectivity**
How selective are your filters? (more selective = faster)
- Latency SLA**
Target response time (e.g., < 100 ms)
- Availability**
Uptime requirement (e.g., 99.9%)

3 Scaling Goals

What are you trying to achieve?

- ✓ Store millions/billions of vectors
- ✓ Keep search fast (low latency)
- ✓ Handle high read/write traffic
- ✓ Support filtering (e.g., tenant, date, tags)
- ✓ Ensure high availability
- ✓ Scale cost-effectively



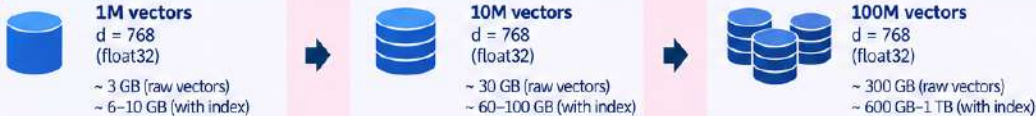
Scale smart, not just big.
Balance performance, cost, and reliability.

4 Scaling Tools & Techniques

These help you scale efficiently.



5 Example: Scaling from 1M to 100M vectors



To scale further:

- Use quantization (e.g., int8)
- Consider smaller dimensions (e.g., 384)
- Shard across multiple nodes
- Replicate for HA
- Use batch upsert + async ingestion
- Monitor and tune for your workload



Key Takeaway:

Scaling a vector database is a combination of **data size**, **system design**, and **smart optimizations**.
Use techniques like quantization, sharding, replication, and batching to meet your performance, availability, and cost goals.



Bigger data.
Smarter systems.
Real impact.



RAG / Vector Search Retrieval Debugging

A step-by-step guide to find where things go wrong and how to fix them.

Find the issue.
Fix the pipeline.
Get better answers!

Common Issues

- 1 Chunking**
Chunks too large/small, losing context.
- 2 Embedding**
Wrong model, low-quality embeddings.
- 3 Query**
Vague or poorly formulated query.
- 4 Top-K**
Too low (misses relevant docs) or too high (adds noise).
- 5 ANN Recall**
Index not tuned, low recall.
- 6 Reranker**
Not used or weak reranker.
- 7 Context / LLM**
Relevant doc retrieved but LLM ignores it (prompting or context window issues).



RAG / Search Quality is Bad
"I'm not getting good relevant answers"



Was the relevant document retrieved?

NO

YES



Check these areas:

- 1 Chunking**
Are chunks meaningful?
- 2 Embedding**
Is the right model used?
- 3 Query**
Is the query well-formed?
- 4 Top-K**
Is K high enough?
- 5 ANN Recall**
Is the index tuned properly?



If the document isn't retrieved, the issue is in the retrieval pipeline (data, embeddings, or search).



Is it ranked correctly (in top-k)?

NO

YES



Check Reranker

- Are you using a reranker?
- Is the reranker model appropriate?
- Are features (query + doc) passed correctly?
- Try a stronger reranker (e.g., BGE, Cohere Rerank).



If relevant doc is retrieved but not in top-k, improve reranking or increase top-k.



Check Context / LLM

- Is the document actually included in the prompt?
- Is the prompt well-designed?
- Is the context too long (important info gets lost)?
- Is the LLM following instructions?
- Try better prompting, summarization, or structured context.



If the right doc is in top-k but answer is still bad, the issue is in the LLM or prompt/context.



Debugging Checklist

- ✓ Inspect retrieved documents manually
- ✓ Check chunk size and content quality
- ✓ Verify embedding model and embedding quality (visualize if needed)
- ✓ Try different query formulations
- ✓ Experiment with different top-K values
- ✓ Check ANN index parameters (recall)
- ✓ Use a reranker and compare results
- ✓ Inspect the final prompt sent to LLM
- ✓ Look at LLM's reasoning (chain-of-thought)
- ✓ Test with known good examples
- ✓ Monitor metrics (recall@k, hit rate, answer relevance)



Useful Tools

- Evaluate retrieval with labeled test queries
- Use embedding visualization (e.g., UMAP, t-SNE)
- Check ANN recall with brute-force search (small set)
- Log and inspect top-K results
- A/B test rerankers and prompts
- Use RAG evaluation frameworks (e.g., RAGAS, TruLens)



Success Metrics

- Retrieval hit rate (relevant doc in top-k)
- Reranked relevance (relevant doc in top-3/top-5)
- End-to-end answer quality (human eval or LLM judge)
- Latency within SLA

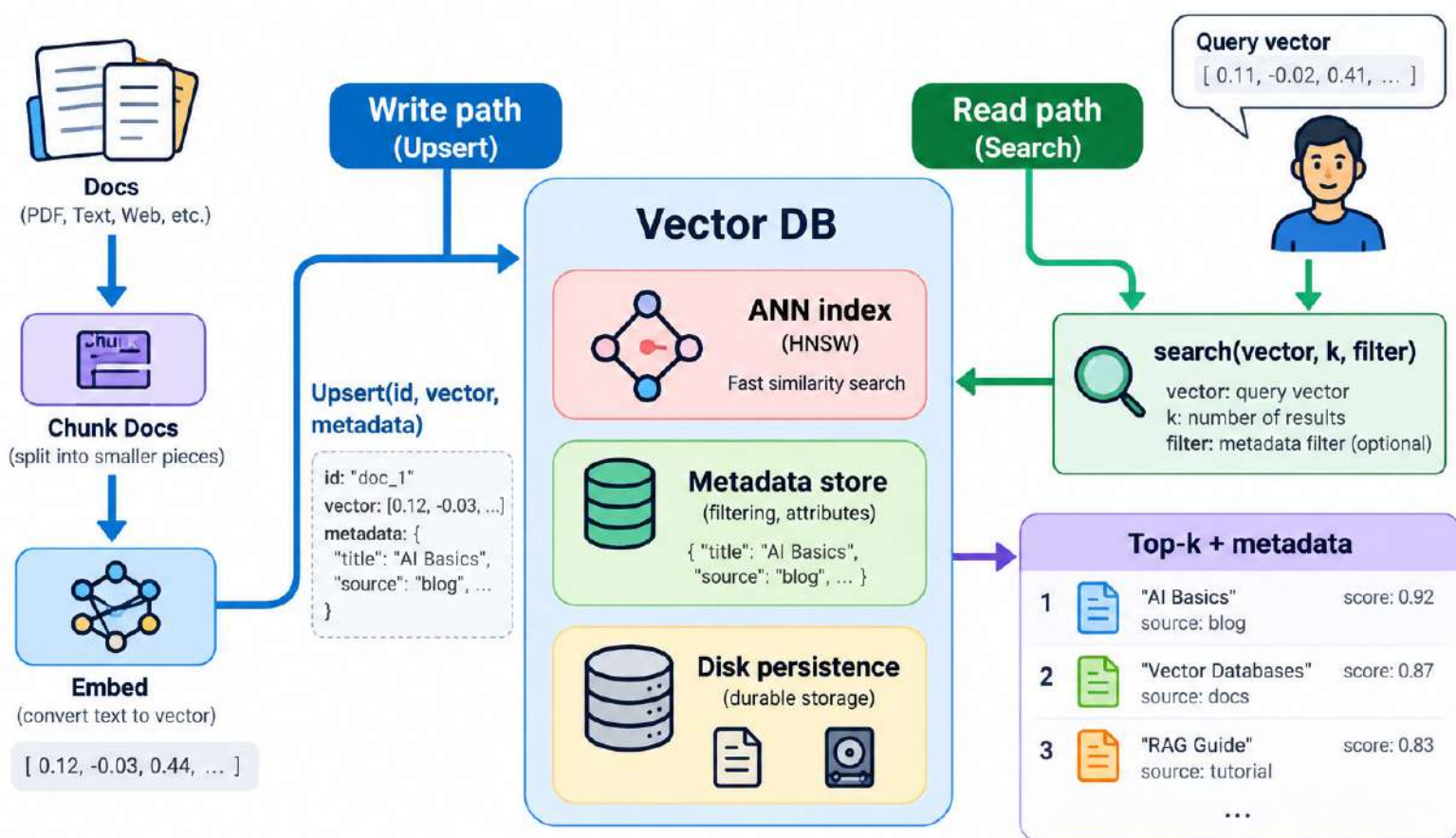


Key Takeaway:

RAG quality issues can usually be traced **step-by-step** — first **retrieval**, then **ranking**, then **LLM/prompt**. Debug systematically, **measure results**, and **fix the right layer**.



Better retrieval.
Better answers.
Happier users.



Store vectors + metadata → Fast filtered similarity search → Persistent & scalable



Query: similar docs WHERE tenant = 'A'
"Find the top-k most similar documents, but only from tenant A"



Post-filter: ANN first, then drop non-A

Search everywhere, then filter by tenant A

1

Run ANN search on all vectors

Get top-k (e.g., k=5) from entire index

2

Filter results by tenant = 'A'

Drop non-A results

3

Final results

May have too few or even zero results

Top-5 from ANN (across all tenants)

1		Doc B1	tenant: B	(score 0.95)
2		Doc B2	tenant: B	(score 0.93)
3		Doc B3	tenant: B	(score 0.91)
4		Doc B4	tenant: B	(score 0.90)
5		Doc B5	tenant: B	(score 0.88)

After filtering (tenant = 'A')

No results!

All top-k were from tenant B



✗ Top-k may be all tenant B → too few / zero results



Vector DB

(vectors + metadata)

	Doc A1	tenant: A
	Doc A2	tenant: A
	Doc A3	tenant: A
...		
	Doc B1	tenant: B
	Doc B2	tenant: B
	Doc B3	tenant: B
...		

Each vector has metadata (e.g., tenant, source, date, ...)

```
{
  "id": "doc_A1",
  "vector": [...],
  "metadata": { "tenant": "A", ... }
}
```



Pre/▲ filter: restrict search to tenant A

Apply filter during search (search only within A)

1

Apply filter first

Restrict search space to tenant = 'A' (only A's vectors)

2

Run ANN search within filtered set

Get top-k from tenant A only

3

Final results

Correct top-k within tenant A

Top-5 from ANN (within tenant A)

1		Doc A1	tenant: A	(score 0.94)
2		Doc A2	tenant: A	(score 0.91)
3		Doc A3	tenant: A	(score 0.87)
4		Doc A4	tenant: A	(score 0.85)
5		Doc A5	tenant: A	(score 0.82)



Correct top-k within A



Key Takeaway: Use pre-filtering (filter during search) to get correct and diverse top-k results within the desired tenant. Post-filtering can return too few or zero results because the initial top-k may be dominated by other tenants.



Code Implementation: Vector Databases

Same idea • Three popular options • From local start to production scale

Store Search Filter Scale

Same concept.
Different tools.
Your choice.

A • Chroma

Local, simplest start



- ✓ Easiest to get started
- ✓ Good for local development / prototypes
- ✓ Embeds for you by default (no need to create embeddings manually)

```
Python
import chromadb
client = chromadb.PersistentClient(path="./db")
col = client.get_or_create_collection("docs")

col.add(
    ids=["d1", "d2"],
    documents=["How to reset password", "Refund policy"],
    metadatas=[{"topic": "account"}, {"topic": "billing"}],
) # Chroma embeds for you by default

res = col.query(
    query_texts=["I forgot my login"],
    n_results=2,
    where={"topic": "account"} # filtered ANN
)

print(res["documents"])
```

Output (example)

```
[
  [ "How to reset password" ]
]
```



B • Qdrant

Open Source, production-ready



- ✓ High performance & scalable
- ✓ Rich filtering (payload)
- ✓ Good for production (self-hosted or managed)

```
Python
from qdrant_client import QdrantClient, models
q = QdrantClient(url="http://localhost:6333")
q.recreate_collection(
    "docs",
    vectors_config=models.VectorParams(
        size=1536,
        distance=models.Distance.COSINE
    ),
)
q.upsert(
    "docs",
    points=[
        models.PointStruct(
            id=1, vector=vec1, payload={"tenant": "A"}
        )
    ]
)
hits = q.search(
    "docs",
    query_vector=qvec,
    limit=5,
    query_filter=models.Filter(must=[
        models.FieldCondition(
            key="tenant",
            match=models.MatchValue(value="A")
        )
    ]) # pre-filtered to tenant A ✓
```

Output (example)

```
[
  {
    "id": 1, "score": 0.94, "payload": {"tenant": "A", ...},
    "id": 7, "score": 0.91, "payload": {"tenant": "A", ...}
  ]
]
```



C • Pinecone

Managed, zero-ops



- ✓ Fully managed (no infrastructure to maintain)
- ✓ Scales automatically
- ✓ Filtering support (metadata)
- ✓ Great for production & teams

```
Python
import os
from pinecone import Pinecone

pc = Pinecone(api_key=os.environ["PINECONE_API_KEY"])
idx = pc.Index("docs")

idx.upsert(
    vectors=[["d1", vec1, {"tenant": "A"}]],
    namespace="prod"
)

res = idx.query(
    vector=qvec,
    top_k=5,
    namespace="prod",
    filter={"tenant": {"$eq": "A"}},
    include_metadata=True
)

print(res["matches"])
```

Output (example)

```
[
  {
    "id": "d1", "score": 0.93, "metadata": {"tenant": "A", ...},
    "id": "d5", "score": 0.89, "metadata": {"tenant": "A", ...}
  ]
]
```



Key Takeaway:

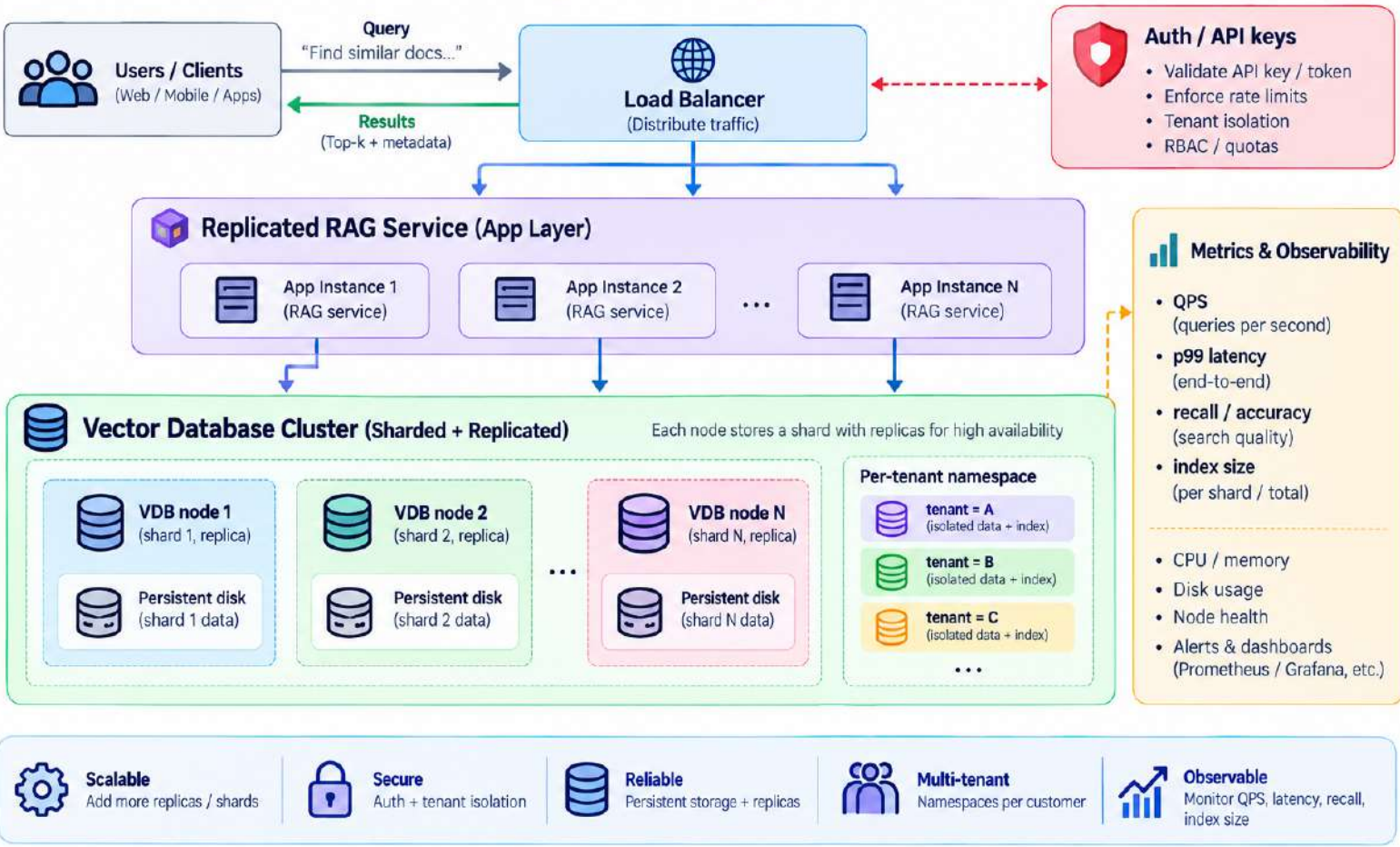
All three do the same core things — store vectors + metadata, and let you search with filters.

Choose based on your needs: Chroma (simple), Qdrant (powerful & production-ready), Pinecone (fully managed).

Build → Test → Scale 🚀

RAG Service with Vector DB – Scalable, Multi-Tenant Architecture

(Replicas • Sharding • Persistent Storage • Observability)



FAISS vs Vector Database

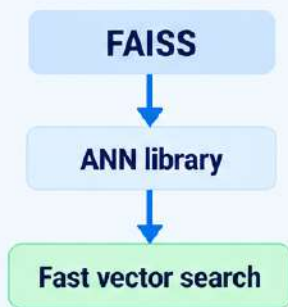
Same core idea (ANN), different scope



FAISS

(Facebook AI Similarity Search)

A high-performance
ANN library



- ✓ C++/Python library (by Meta)
- ✓ Implements ANN algorithms (e.g., IVF, HNSW, PQ, LSH)
- ✓ Extremely fast similarity search
- ✓ In-memory (mostly)
- ✓ No built-in metadata or filtering
- ✓ You handle storage, scaling, APIs, etc.
- ✓ Great for research, prototypes, custom systems

Simple FAISS flow



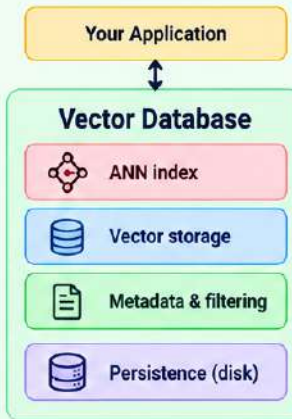
VS



Vector Database

A complete database for vector data

Beyond search –
production ready



- ✓ Built-in ANN index (HNSW, IVF, PQ, etc.)
- ✓ Stores vectors + metadata
- ✓ Filtering (e.g., tenant, date, tags)
- ✓ Persistence (durable storage)
- ✓ Replication & high availability
- ✓ Sharding (horizontal scaling)
- ✓ Auth / API keys / access control
- ✓ Client APIs (REST, gRPC, SDKs)
- ✓ Monitoring & observability
- ✓ Ready for production (multi-tenant, backup, upgrades, etc.)

Feature	FAISS	Vector Database
Target	Research / fast similarity search	Production-ready vector data management
What it is	ANN library (C++/Python)	Full-fledged database
Stores metadata	✗ No (you manage it)	✓ Yes
Persistence	✗ No (in-memory by default)	✓ Yes (disk-based, durable)
Filtering	✗ No (basic only, you implement)	✓ Yes (rich filtering)
Scaling / HA	✗ No (you build it)	✓ Yes (replication, sharding, clustering)
APIs / Auth / Monitoring	✗ No (you build it)	✓ Yes (built-in)
Best use case	Prototypes, research, custom systems	Real-world applications (RAG, search, multi-tenant, etc.)



Key Takeaway

FAISS is the engine
(for fast ANN search).

A Vector Database is the car
(engine + storage + metadata +
filtering + scaling + security +
everything you need to run
in production).

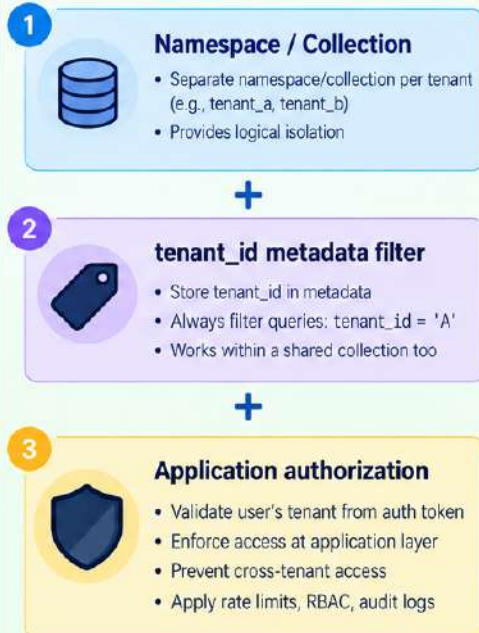
Multi-Tenant Security in Vector Databases

Right data. Right tenant. Always.

Tenant A Query Flow



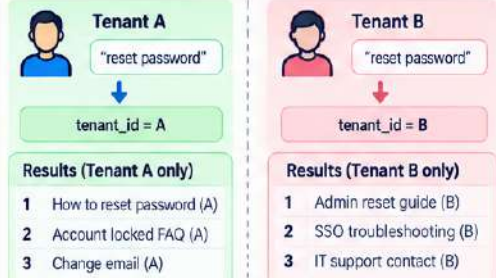
How to Implement Multi-Tenant Security



Why It Matters?

- Prevent data leakage**
Tenant A should never see Tenant B's data (even if embeddings are similar).
- Compliance & trust**
Required for enterprise use (SOC2, GDPR, etc.).
- Safe at scale**
Works with thousands of tenants.

Example: Same Query, Different Tenants



Relevance ≠ authorization.

A document may be highly relevant, but if it belongs to another tenant, it must not be returned.



Use all three layers together:

namespace /
collection

+

tenant_id
metadata filter

+

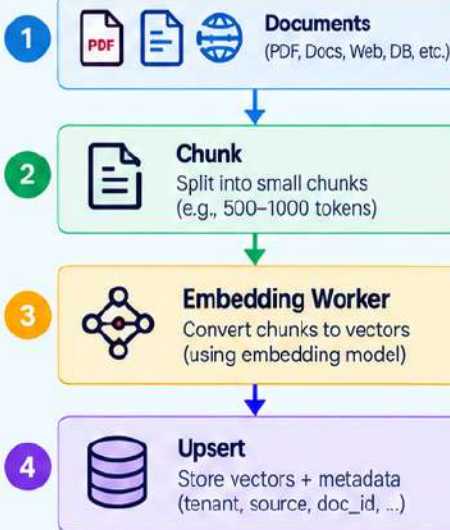
application
authorization

Production Architecture for RAG with Vector Database

From documents to answers — at scale

Document Ingestion Pipeline

Ingest → Chunk → Embed → Upsert

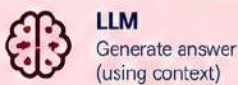
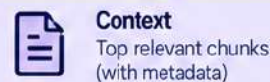


Production Considerations

- Scalability**
Sharding, replication, autoscaling
- Security**
Auth / API keys, tenant isolation, RBAC
- Observability**
QPS, p99 latency, recall, index size, logs & alerts
- Reliability**
Backups, disaster recovery
- Multi-tenancy**
Namespaces / collections + metadata filters + application authorization

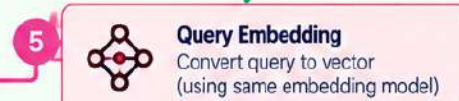
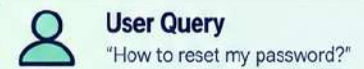
Vector Database

Store, index and search vector data



Query Pipeline

Query → Embed → Search → Rerank → Answer



End-to-End Flow (Animated Steps)

- 1 Ingest**: Load documents from various sources
- 2 Chunk**: Split documents into small chunks
- 3 Embed**: Convert chunks to vector embeddings
- 4 Upsert**: Store vectors + metadata in Vector DB
- 5 Query Embed**: Convert user query to vector
- 6 Filter**: Apply metadata filters (e.g., tenant, date)
- 7 ANN**: Run similarity search (e.g., HNSW)
- 8 Top-K**: Get top-k candidate documents
- 9 Rerank**: Rerank for better relevance
- 10 Context**: Prepare final context for LLM
- 11 LLM**: Generate answer using context
- 12 Answer**: Return final response (with citations)

Relevance ≠ authorization.
Always enforce tenant isolation & access control.



Key Takeaway:

A production-ready RAG system combines a vector database for fast, filtered search with a robust ingestion and query pipeline, security, observability, and scalability.



Production Metrics for Vector DB & RAG Systems

Measure what matters • Go beyond averages • Keep your RAG system fast, reliable, and relevant

Observe → Optimize → Scale

Good metrics lead to better user experiences!



QPS
Queries per second

520 ↑ 12%



p50 Latency
(ms)

120 ↓ 8%



p95 Latency
(ms)

380 ↓ 10%



p99 Latency
(ms)

780 ↓ 15%



Error Rate
(%)

0.8% ↓ 50%



Recall@K
Search quality

0.92 ↑ 6%



Index Size
(GB)

320 ↑ 10%



Memory Usage
(GB)

28 / 64 44%



CPU Usage
(%)

46%



Write Throughput
(vectors/sec)

1,200 ↑ 18%

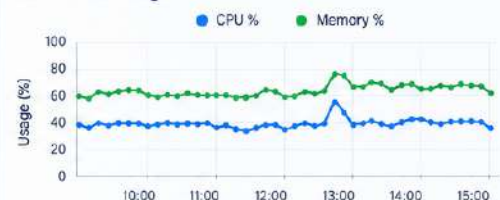
Latency Distribution (p50 / p95 / p99)



QPS and Error Rate



Resource Usage



End-to-End RAG Latency Breakdown

User-perceived latency = sum of all components



Key Takeaways

- ✓ Monitor p95 and p99, not just average latency.
- ✓ Track both system metrics (CPU, memory, index size) and business metrics (QPS, recall@K).
- ✓ Keep error rate low and alert on spikes.
- ✓ Measure end-to-end latency (Embedding + Search + Rerank + LLM).
- ✓ Set up dashboards and alerts (Prometheus + Grafana, etc.).
- ✓ Continuously optimize for better user experience.

"You can't improve what you don't measure."



Goal: Fast + Relevant + Reliable + Scalable RAG in Production

Metrics turn guesswork into growth.



Monitor → Optimize → Delight Users

Vector Database – Final Decision Tree & Cheat Sheet

Choose the right tool and approach for your use case

1 Choose the Right Tool (Based on Workload)

 How big is the workload?

Tiny

(small dataset / prototype)



Just a prototype?

Yes



Chroma

- Easiest to start
- Local / lightweight
- Good for prototyping



Want full library control?

Yes



FAISS

- ANN library (by Meta)
- Very fast
- No built-in metadata
- You handle storage, etc.

Moderate

(growing data / team usage)



Already using Postgres?

Yes



pgvector

- Extension for Postgres
- Great if you already use PostgreSQL
- SQL + vector search



Need a dedicated vector service?

Yes



Vector DB

- e.g., Qdrant, Weaviate
- Built-in metadata, filtering, APIs
- Good for production without heavy ops

Large

(millions of vectors / production)



Zero ops (prefer managed)?

Yes



Managed Vector DB

- e.g., Pinecone, Azure AI Search, etc.
- Fully managed
- Highly scalable
- No infrastructure maintenance



Need OSS + filtering?

Yes



Qdrant

- Open source
- High performance
- Great filtering
- Production-ready



Need OSS + hybrid search + built-in modules?

Yes



Weaviate

- Open source
- Hybrid search (vector + keyword)
- Built-in modules (e.g., generative, transformers)

2 What to Use When? (Quick Guide)



Meaning?

(semantic search)



Embeddings

(e.g., OpenAI, BGE, etc.)



Exact match?

(keyword search)



BM25 / Traditional DB

(e.g., Elasticsearch, Postgres)



Large vector corpus?

(fast similarity search)



ANN

(e.g., HNSW, IVF, PQ)



Need exact + meaning? →

Hybrid Search

(Dense + Sparse)



Best ranking?

(improve top-k quality)



Reranker

(e.g., cross-encoder, Cohere)



Tenant / date / security constraint?



Filter during retrieval

(e.g., metadata filters, namespace)



High availability (HA)? →

Replication

(multiple replicas)



Horizontal capacity? →

Sharding

(split data across nodes)



Key Takeaway:

"Pick the simplest tool that meets your current needs, and make sure it can scale with your future requirements."



Start simple



Use filtering for multi-tenant and security



Monitor and scale when needed



There is no one-size-fits-all.

The best choice depends on your data size, team, infrastructure, and long-term goals.